

Report: Robust SD Card Data Logging System using ESP32-S3

-By: Pranav R Nair

1. Introduction

This project implements a **Fault-Tolerant Data Logging System** using the ESP32-S3 microcontroller and a standard MicroSD card. Unlike basic file writers, this system includes "self-healing" capabilities to automatically fix filesystem corruption, ensure required directory structures exist, and utilize FreeRTOS multitasking to prevent data write operations from blocking the main sensor acquisition loop.

2. Background

- **Hardware Platform:** ESP32-S3 DevKitC-1
- **Storage Medium:** SPI-based MicroSD Card (FAT32 filesystem).
- **Communication Protocol:** **SPI** (Serial Peripheral Interface) is used for high-speed data transfer between the microcontroller and the storage card.
- **Software Framework:** ESP-IDF (Espressif IoT Development Framework) using standard C file handling (`stdio.h`) and FreeRTOS for task management.

3. Methodology

A. Hardware Setup

The SD Card Module is connected to the ESP32-S3 via the SPI bus. To ensure stability, the module is powered by **5V** (to feed the onboard voltage regulator) while logic remains at 3.3V.

- **CS (Chip Select):** GPIO 10
- **MOSI (Data Out):** GPIO 11
- **CLK (Clock):** GPIO 12
- **MISO (Data In):** GPIO 13

B. Software Architecture

1. **Self-Healing Initialization:**

- On startup, the system attempts to mount the card.
 - If the mount fails (due to corruption), the flag `.format_if_mount_failed = true` triggers an automatic format to FAT32.
 - The system scans for required folders (`/data`, `/config`). If missing, it uses `mkdir()` to create them.
2. **Queue-Based Logging (Non-Blocking):**
- **Task 1 (Producer):** The main loop collects data and pushes it into a **FreeRTOS Queue**. This takes microseconds.
 - **Task 2 (Consumer):** A separate background task waits for the Queue. When data arrives, it handles the slow process of opening the file, writing, and saving.

4. Experiment

The system was tested under the following conditions:

1. **Boot Test:** The ESP32 was booted with a blank, unformatted SD card to verify the "Self-Healing" formatting logic.
2. **Write Stress Test:** The main loop was set to generate dummy sensor logs ("Timestamp: X") every 1000ms.
3. **Read Verification:** A text file named `file.txt` was placed in the `/data` folder to verify the ESP32's ability to read external configuration files.
4. **Wiring Fault Simulation:** The MISO line was disconnected to observe error reporting (`ESP_ERR_TIMEOUT`).

5. Results

A. Initialization Phase

The Serial Monitor output confirmed successful mounting and directory creation:

```
I (863) SD_FIXER: SD Card Mounted Successfully! I (903)
SD_FIXER: SUCCESS: Created folder /sdcard/data
```

B. Read Operation

The system successfully opened an existing text file and printed its content to the console, verifying the `read_specific_file()` function:

```
----- FILE CONTENT START ----- Hello! This text is
coming from the SD Card. ----- FILE CONTENT END
-----
```

C. Write Operation

Data was successfully logged to `log.txt` without blocking the main CPU. Timestamps verified that the logging interval remained stable at 1000ms, regardless of SD card write latency.

```
I (25359) SD_FIXER: Saved: /sdcard/data/log.txt
```

6. Conclusion

The implemented code provides a robust foundation for any data-logging IoT device. By decoupling the data generation (sensors) from data storage (SD card) using Queues, the system achieves high reliability. The added "Self-Healing" features ensure that the device can recover from filesystem errors without user intervention, making it suitable for remote field deployments.

7. Outputs (Logs)

Below is a snapshot of the system output confirming successful operation:

```
PROBLEMS 14 OUTPUT TERMINAL DEBUG CONSOLE PORTS XRTOS SERIAL MONITOR MEMORY
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - Silicon Labs CP210x USB to UART Bridge (COM3) Baud rate 1152
Line ending None ▶ Start Monitoring
---- Opened the serial port COM3 ----
I (86359) SD_FIXER: Saved: /sdcard/data/log.txt
I (87359) SD_FIXER: Saved: /sdcard/data/log.txt
I (88359) SD_FIXER: Saved: /sdcard/data/log.txt
I (89359) SD_FIXER: Saved: /sdcard/data/log.txt
I (90359) SD_FIXER: Saved: /sdcard/data/log.txt
I (90419) SD_FIXER: Saved: /sdcard/config/settings.cfg
I (91359) SD_FIXER: Saved: /sdcard/data/log.txt
I (92359) SD_FIXER: Saved: /sdcard/data/log.txt
I (93359) SD_FIXER: Saved: /sdcard/data/log.txt
I (94359) SD_FIXER: Saved: /sdcard/data/log.txt
I (95359) SD_FIXER: Saved: /sdcard/data/log.txt
I (95419) SD_FIXER: Saved: /sdcard/config/settings.cfg
I (96359) SD_FIXER: Saved: /sdcard/data/log.txt
I (97359) SD_FIXER: Saved: /sdcard/data/log.txt
I (98369) SD_FIXER: Saved: /sdcard/data/log.txt
I (99359) SD_FIXER: Saved: /sdcard/data/log.txt
I (100359) SD_FIXER: Saved: /sdcard/data/log.txt
I (100419) SD_FIXER: Saved: /sdcard/config/settings.cfg
I (101359) SD_FIXER: Saved: /sdcard/data/log.txt
I (102359) SD_FIXER: Saved: /sdcard/data/log.txt
I (103359) SD_FIXER: Saved: /sdcard/data/log.txt
I (104359) SD_FIXER: Saved: /sdcard/data/log.txt
I (105359) SD_FIXER: Saved: /sdcard/data/log.txt
I (105419) SD_FIXER: Saved: /sdcard/config/settings.cfg
I (106359) SD_FIXER: Saved: /sdcard/data/log.txt
I (107359) SD_FIXER: Saved: /sdcard/data/log.txt
I (108359) SD_FIXER: Saved: /sdcard/data/log.txt
I (109359) SD_FIXER: Saved: /sdcard/data/log.txt
I (110359) SD_FIXER: Saved: /sdcard/data/log.txt
I (110419) SD_FIXER: Saved: /sdcard/config/settings.cfg
I (111359) SD_FIXER: Saved: /sdcard/data/log.txt
```